

AI Usage Note

Project: Customer Outage Comms Drafter

Overview

This project uses Artificial Intelligence both during the development process and at runtime to generate customer-facing outage communications from technical incident timelines. The goal was to reduce the time required for incident response teams to draft clear, professional, and consistent communication updates during service disruptions.

AI Tools Used

Tool	Role
ChatGPT (OpenAI)	Prompt engineering, project planning, documentation, and code assistance
Antigravity (Google)	Code optimization, debugging support, and project setup guidance
Groq API + Llama 3.3 70B	Runtime AI model used to generate outage communications
GitHub Copilot	Inline code suggestions and development productivity assistance

What AI Helped With

AI played a significant role throughout the project lifecycle.

1. Prompt Engineering

ChatGPT was used to design and refine prompts that instruct the language model to generate:

- Initial Customer Updates
- Progress Updates
- Resolved Updates
- Internal Incident Summaries

The prompts were iteratively improved to ensure the generated messages remained professional, customer-friendly, and aligned with the selected severity and tone.

2. Application Development

AI-assisted coding tools helped with:

- Flask application structure
- API integration with Groq
- JSON response handling
- Error handling logic
- Output parsing and formatting

GitHub Copilot also provided inline code suggestions that accelerated development.

3. Documentation

AI was used to assist in creating:

- Project README
- Setup Instructions
- Architecture Description
- Demo Presentation Script
- AI Usage Documentation

4. Runtime Communication Generation

The Groq API with the Llama 3.3 70B model serves as the core AI engine of the application. It transforms technical incident timelines into customer-friendly communications that can be used during outage situations.

What AI Got Wrong

Although AI significantly accelerated development, some outputs required manual review and correction.

Examples include:

- Generating responses that were too technical for customer communication.
- Producing inconsistent output formats that were difficult to parse programmatically.
- Occasionally ignoring formatting instructions specified in the prompt.

- Suggesting overly complex implementations for simple requirements.
- Generating generic communication messages that lacked context from the provided timeline.

These issues were resolved through prompt refinement, testing, and manual validation before integration.

Best Prompts Used

Prompt 1 – Outage Communication Generation

"Act as an outage communication specialist. Convert the provided technical incident timeline into customer-friendly communications. Generate an Initial Update, Progress Update, Resolved Update, and Internal Incident Summary. Use professional language and adapt the communication style based on severity and tone."

Prompt 2 – Customer-Friendly Messaging

"Rewrite the incident information in language that customers can easily understand. Avoid exposing internal technical details while maintaining transparency and professionalism."

Prompt 3 – Internal Incident Summary

"Create a concise internal incident summary using bullet points. Include timeline highlights, investigation details, impact, resolution actions, and final outcome."

Conclusion

AI significantly improved both the development process and the functionality of the Customer Outage Comms Drafter. It accelerated coding, documentation, and prompt design while enabling dynamic generation of outage communications through a Large Language Model. All AI-generated outputs were reviewed and validated to ensure accuracy, usability, and compliance with project requirements.